ASTANA IT UNIVERSITY · SHP-2216 SOFTWARE DESIGN PATTERNS · ASSIGNMENT 06

Visitor Pattern: Advanced File System

Author: Adilzhan Kuandykov Java 17 · Maven · JUnit 5

Repository: github.com/adilzhandev/AITU-DAA-A6-Visitor

1 Purpose and design choices

The project is an in-memory hierarchical file system with three independent operations over one structure: total size, filtered search and statistics. Elements know only how to store data and protect the tree. Each operation lives in its own visitor, so behaviour can grow without changing the element classes. The UML diagram is on the last page.

Dispatch step 1: `element.accept(visitor)`
Java selects the method by the runtime type of the element.

Dispatch step 2: `visitor.visit(this)`
The compiler selects the overload from the static type of `this`.

Requirement	Implementation
R1 Object model	`Element` is implemented by `FileNode` (name, size) and `DirectoryNode` (named children). Nesting is arbitrary.
R2 Double dispatch	Each element's `accept` is the single line `visitor.visit(this)`.
R3 Size	`TotalSizeVisitor` adds file sizes only. Directories contribute nothing of their own.
R4 Search	`SearchVisitor` takes a `Predicate<String>` name criterion and an inclusive minimum size. Both must hold.
R5 Statistics	`StatisticsVisitor` reports file count, directory count (root included) and maximum depth (root = 0).
R6 Integrity	`DirectoryNode.add` rejects null children, a second parent, duplicate sibling names and cycles with `IllegalArgumentException`, before any state changes. The `FileNode` constructor rejects negative sizes.
R7 Traversal	Visitors own traversal: `visit(DirectoryNode)` calls `child.accept(this)` on every child.
R8 Extensibility	Commit `0f4c160` adds `StatisticsVisitor` and its test only. No file in `model/` changes.

Controlled state. Visitor instances are reusable. Each one tracks a `depth` counter, and a `visit` call at depth 0 starts a new run and clears earlier results. Results therefore describe the latest run only, whichever order runs happen in. Instances are not thread-safe.

Paths and depth. Search paths are relative to the element on which `accept` was called. The root's name is omitted, and `/` joins the rest. A file used as the root reports its own name. A `TreeSet` sorts the full path strings, so `src-old/App.java` precedes `src/App.java` because `-` sorts before `/`. Depth starts at 0 on the root and grows by 1 per level. Maximum depth includes files and empty directories.

Type safety. The code contains no `instanceof`, `getClass()` branching, reflection or casts. `AbstractElement` has a package-private constructor and `attachTo`, so parent links can only be set through `DirectoryNode.add`.

2 Clean Code practices

Practice	Where	Justification
Meaningful naming	`SearchVisitor.minimumSize`, `DirectoryNode.isSelfOrAncestor`	The names state the rule being applied, so the code reads clearly without comments.
Single Responsibility	`TotalSizeVisitor`, `DirectoryNode`	The visitor only sums sizes and the directory only maintains its children, so each has one reason to change.
Small methods	`DirectoryNode.add`, `SearchVisitor.matches`	`add` has three steps after delegating validation, and `matches` holds the whole filter in one expression.
Encapsulation	`DirectoryNode.getChildren`, `AbstractElement.attachTo`	An immutable copy and a package-private setter mean that tree invariants cannot be bypassed from outside.
No type-checking chains	`FileNode.accept`, `DirectoryNode.accept`	Overload selection through `visitor.visit(this)` replaces every possible `if/instanceof` branch.

3 Sample output

Output of `mvn -q compile exec:java` on the project's own 13-element `campus-project` tree. Two consecutive runs produced identical output.

```
Total size: 63656 bytes
Search name ends with '.java' and size >= 2048:
  src-old/App.java
  src/App.java
  src/util/Parser.java
Files: 7
Directories: 6
Maximum depth: 3
Total size after running the same visitor twice: 63656 bytes
Rejected change: Duplicate sibling name: README.md
```

4 Test summary

38	0	0	0	BUILD SUCCESS
tests run	failures	errors	skipped	mvn test

Test class	Tests	Checks	Focus
`ElementDispatchTest`	3	T1	A recording visitor proves that each element reaches its own overload.
`TotalSizeVisitorTest`	6	T2, T6, T7	Nested sums, subtree roots, an empty directory, repeat runs.
`SearchVisitorTest`	12	T3, T4, T6, T7	Combined filters, inclusive threshold, path order, no matches, recovery after a failed run.
`StatisticsVisitorTest`	6	T5, T6, T7	Counts, depth through an empty directory, a smaller tree after a larger one.
`TreeIntegrityTest`	11	T8	Each invalid mutation is rejected and leaves the tree unchanged. Valid deep nesting works.

5 Traversal and extension trade-offs

- **Visitor-owned traversal** keeps the elements minimal and lets each visitor track the path or depth it needs. The cost is one repeated loop per visitor, and a new visitor could forget to recurse. A separate walker would remove that loop, but it would have to pass path and depth information into every visitor, which is more code for three operations.
- **Adding an operation** is cheap and closed for modification: one new class implements `Visitor`, and the elements stay untouched.

- **Adding an element type** is expensive: a `SymlinkNode`, for example, needs a new `visit` overload in `Visitor` and an implementation in every existing visitor. The compiler lists each place that must change, so nothing is missed, but the change spreads across the whole hierarchy. Visitor fits this project because the element types are stable while operations grow.
- **A recursion limit** applies as well: traversal depth is bounded by the JVM call stack, which is acceptable for an in-memory file tree.

6 Repository and references

GitHub: https://github.com/adilzhandev/AITU-DAA-A6-Visitor. The README gives the exact commands to run the tests and the application.

Course Lecture 9 (Visitor Pattern, double dispatch); E. Freeman and E. Robson, *Head First Design Patterns*, 2nd ed.; Refactoring.Guru, Visitor.

UML class diagram

